

Database Schema

Project Name

CarbonWise

Version: 1.0

Database: PostgreSQL 16

ORM: Prisma ORM

Status: Approved

1. Purpose

This document defines the database architecture for CarbonWise.

The database is designed to support the three core objectives of the challenge:

Understand

↓

Track

↓

Reduce

The schema prioritizes:

- Data Integrity
 - Scalability
 - Query Performance
 - Security
 - Maintainability
-

2. Database Architecture Overview

```
graph TD; Users --- UserPreferences; Users --- Activities; Users --- CarbonRecords; Users --- Recommendations; Users --- RecommendationActions; Users --- Goals; Users --- GoalMilestones; Users --- CarbonTwinScenarios; Users --- Notifications;
```

A diagram showing the database architecture. A central vertical line represents the 'Users' entity. From this line, ten horizontal lines branch out to the right, each connected to a table name: 'UserPreferences', 'Activities', 'CarbonRecords', 'Recommendations', 'RecommendationActions', 'Goals', 'GoalMilestones', 'CarbonTwinScenarios', and 'Notifications'.

3. Entity Relationship Overview

```
graph TD; User --- Activities; User --- CarbonRecords; User --- Recommendations; User --- Goals; User --- CarbonTwinScenarios; User --- Notifications;
```

A diagram showing the entity relationship. A central vertical line represents the 'User' entity. From this line, six horizontal lines branch out to the right, each connected to a table name: 'Activities', 'CarbonRecords', 'Recommendations', 'Goals', 'CarbonTwinScenarios', and 'Notifications'.

4. Users Table

Purpose

Stores user account information.

Prisma Model

```
model User {
  id          String    @id @default(uuid())
  fullName    String
  email       String    @unique
  passwordHash String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  preferences UserPreference?
  activities   Activity[]
  carbonRecords CarbonRecord[]
  recommendations Recommendation[]
  goals        Goal[]
}
```

Fields

Field	Type	Description
id	UUID	Primary Key
fullName	String	User Name
email	String	Unique Email
passwordHash	String	bcrypt Hash
createdAt	DateTime	Creation Date
updatedAt	DateTime	Last Update

5. User Preferences Table

Purpose

Stores personalization settings.

Prisma Model

```
model UserPreference {  
  id                String @id @default(uuid())  
  userId            String @unique  
  
  dietType          String  
  transportationType String  
  sustainabilityGoal String  
  
  user              User @relation(fields:[userId], references:[id])  
}
```

Examples

```
Vegetarian  
  
Public Transport  
  
Reduce 20%
```

6. Activities Table

Purpose

Stores all user activities contributing to emissions.

Prisma Model

```
model Activity {
  id          String   @id @default(uuid())

  userId      String

  category     ActivityCategory

  activityName String

  quantity     Float

  unit         String

  activityDate DateTime

  createdAt    DateTime @default(now())

  user        User @relation(fields:[userId], references:[id])
}
```

Activity Categories

```
enum ActivityCategory {
  TRANSPORTATION
  FOOD
  ENERGY
  SHOPPING
}
```

Examples

Transportation:

```
Car Travel
Flight
```

Bus
Bike

Food:

Meat Meal
Vegetarian Meal
Dairy Consumption

Energy:

Electricity
Heating
Cooling

Shopping:

Clothing
Electronics
Household Goods

7. Carbon Records Table

Purpose

Stores calculated emissions.

Prisma Model

```
model CarbonRecord {  
  id          String    @id @default(uuid())  
  
  userId      String
```

```

    activityId      String?
    category        ActivityCategory
    emissionValue   Float
    emissionUnit    String
    createdAt       DateTime @default(now())
    user            User @relation(fields:[userId], references:[id])
  }

```

Example

Transportation

45 kg CO₂e

8. Recommendations Table

Purpose

Stores personalized recommendations generated by the Recommendation Engine.

Prisma Model

```

model Recommendation {
  id          String @id @default(uuid())
  userId      String
  title       String
  description  String
  priority    RecommendationPriority
}

```

```

    estimatedSavings    Float

    accepted            Boolean @default(false)

    createdAt           DateTime @default(now())

    user                User @relation(fields:[userId], references:[id])
  }

```

Priority Enum

```

enum RecommendationPriority {
  HIGH
  MEDIUM
  LOW
}

```

Example

```

Use Public Transport Twice Weekly

Estimated Savings:

120 kg CO2/year

```

9. Recommendation Actions Table

Purpose

Tracks whether recommendations were followed.

Prisma Model

```

model RecommendationAction {
  id          String @id @default(uuid())

```



```
    recommendationId    String
    actionTaken          Boolean
    completedAt          DateTime?
  }
```

Why This Exists

Supports:

```
Track
and
Reduce
```

portions of the challenge.

10. Goals Table

Purpose

Stores sustainability goals.

Prisma Model

```
model Goal {
  id          String @id @default(uuid())
  userId      String
  title       String
  targetReduction  Float
  currentProgress  Float
}
```

```

    targetDate      DateTime

    status          GoalStatus

    createdAt       DateTime @default(now())

    user            User @relation(fields:[userId], references:[id])
  }

```

Goal Status

```

enum GoalStatus {
  ACTIVE
  COMPLETED
  PAUSED
}

```

Example

```

Reduce Transportation Emissions

Target:

20%

```

11. Goal Milestones Table

Purpose

Tracks progress.

Prisma Model

```

model GoalMilestone {
  id          String @id @default(uuid())

```

```
goalId      String
title       String
targetPercentage Int
completed   Boolean
}
```

Example

```
25%
50%
75%
100%
```

12. Carbon Twin Scenario Table

Purpose

Stores simulation results.

Prisma Model

```
model CarbonTwinScenario {
  id          String @id @default(uuid())
  userId      String
  scenarioName String
  currentEmission Float
  projectedEmission Float
}
```

```
    potentialSavings      Float
    createdAt             DateTime @default(now())
}
```

Example

Current:

250 kg CO₂

Projected:

180 kg CO₂

Savings:

70 kg CO₂

Why Important

This table powers:

Personalized Insights

which is a key challenge requirement.

13. Notifications Table

Purpose

Stores reminders and alerts.

Prisma Model

```
model Notification {
  id          String @id @default(uuid())

  userId      String

  title       String

  message     String

  isRead      Boolean @default(false)

  createdAt   DateTime @default(now())
}
```

Examples

Goal Milestone Reached

New Recommendation Available

Weekly Carbon Report

14. Emission Factors Table

Purpose

Stores official emission factors.

Prisma Model

```
model EmissionFactor {
  id          String @id @default(uuid())

  category    String
}
```

```
    itemName      String
    factor         Float
    unit           String
    source         String
}
```

Example

Car Travel

0.192 kg CO₂/km

15. Indexing Strategy

High Priority Indexes

Users

```
CREATE UNIQUE INDEX idx_users_email
ON users(email);
```

Activities

```
CREATE INDEX idx_activities_user
ON activities(user_id);
```

Carbon Records

```
CREATE INDEX idx_carbon_records_user
ON carbon_records(user_id);
```

Recommendations

```
CREATE INDEX idx_recommendations_user  
ON recommendations(user_id);
```

Why

Supports:

Efficiency (Medium Impact)

evaluation criteria.

16. Query Optimization Strategy

Pagination

Applied to:

- Activities
- Recommendations
- Notifications

Example:

```
LIMIT 20  
OFFSET 0
```

Selective Queries

Avoid:

```
SELECT *
```

Use:

```
SELECT id, title
```

17. Security Considerations

Supports:

Security (Medium Impact)

Password Storage

Never store:

Plain Text Passwords

Store:

bcrypt Hashes

SQL Injection Protection

Provided by:

Prisma ORM

Sensitive Data

Never store:

- Access Tokens

- API Keys
- Secrets

inside database records.

18. Backup Strategy

Daily

Incremental Backup

Weekly

Full Backup

Monthly

Archive Backup

19. Scalability Strategy

Future tables may support:

- Community Challenges
- AI Carbon Coach
- Sustainability Reports
- Smart Device Data

without major schema redesign.

20. Requirement Traceability

Understand

Supported By:

- Activities
- Carbon Records

Track

Supported By:

- Goals
- Milestones
- Notifications

Reduce

Supported By:

- Recommendations
- Recommendation Actions
- Carbon Twin Scenarios

21. Success Criteria

The database design is successful when:

- ✓ User data remains secure
- ✓ Carbon calculations are traceable
- ✓ Recommendations are auditable
- ✓ Queries remain performant
- ✓ Goal tracking works reliably
- ✓ Carbon Twin simulations are stored efficiently
- ✓ Future features can be added without major schema changes